04-docker-realtime.md

🧠 Big Picture (MOST IMPORTANT)

Dockerfile = Recipe 📖
Image = Packed food box 📦
Container = Eating the food 🍽️

👉 You:

1. Write recipe (Dockerfile)
2. Cook & pack it (docker build → image)
3. Eat/run it (docker run → container)

🍳 Step 1 — Basic Dockerfile (ELI5)

FROM ubuntu
RUN touch mustafa.txt

🧠 Think like this:

FROM → Which kitchen? (Ubuntu kitchen)
RUN → What to cook? (create file)

👉 Result:

- Image has `mustafa.txt`
- When you run → container has that file

📦 Step 2 — COPY

COPY aws.txt aws.txt

🧠 Real-world:

COPY = Put your local file into lunch box

👉 You must have file in your laptop first

📦 Step 3 — ADD

ADD devops.pdf devops.pdf

🧠 Real-world:

ADD = COPY + extra power

👉 It can:

- copy local file ✅
- download from URL ✅
- extract zip/tar automatically ✅

⚠️ Easy rule:

Use COPY always
Use ADD only if you need URL or auto-extract

🌍 Step 4 — ADD from URL

ADD https://...tomcat.zip .

🧠 Real-world:

ADD = download file from internet into your box

👉 Inside container → file will be there

📁 Step 5 — WORKDIR

WORKDIR /mustafa
RUN touch harish.txt

🧠 Real-world:

WORKDIR = Change working folder

👉 Like:

cd /mustafa
touch harish.txt

🔥 Step 6 — RUN vs CMD (VERY IMPORTANT)

RUN

RUN yum install git -y

🧠 Real-world:

RUN = cook during preparation

👉 Happens during `docker build`

CMD

CMD yum install maven -y

🧠 Real-world:

CMD = what happens when you open the box

👉 Happens during `docker run`

🔥 Easy memory trick:

RUN → build time
CMD → run time

⚠️ Important rule

Multiple RUN → OK
Multiple CMD → only last one works

🚀 Step 7 — ENTRYPOINT

ENTRYPOINT ["yum", "install", "git", "-y"]

🧠 Real-world:

ENTRYPOINT = fixed behavior (cannot change easily)
CMD = default behavior (can override)

🔥 Difference (simple)

Type	Can override?	Meaning
CMD	YES	default action

Type	Can override?	Meaning
ENTRYPOINT	NO	forced action

Example:

docker run image tree

👉 If CMD → replaced
👉 If ENTRYPOINT → added to command

🌱 Step 8 — ENV vs ARG

ARG

ARG cloud=aws

🧠 Real-world:

ARG = temporary variable (only while cooking)

👉 Only during build

ENV

ENV course=devops

🧠 Real-world:

ENV = permanent label on box

👉 Available inside container

🔥 Difference

Feature	ARG	ENV
Available at build	YES	YES
Available at runtime	NO	YES
Can override	YES	harder

💣 Brutal truth (you need this)

You memorized:

commands

But you didn't understand:

lifecycle (build vs run)

👉 That's why confusion came in RUN vs CMD vs ENTRYPOINT

🧠 Final cheat sheet (remember this only)

FROM → base OS
RUN → build time work
COPY → copy local files
ADD → copy + download
WORKDIR → change directory
CMD → default run command
ENTRYPOINT → fixed run command
ARG → build variable
ENV → runtime variable

🚀 One-line memory trick

Build → RUN, ARG
Run → CMD, ENTRYPOINT, ENV

🎯 If you want next level

I can give you:

- real-world Node.js Dockerfile (production ready)
- Docker + NGINX + PM2 setup
- multi-stage builds (important for interviews)

Just say: **"give real project dockerfile"**

ELI5 AND ELI10 AS I GIVEN ABOVE

You're close, but your notes mix steps with concepts. I'll clean it up so you can **remember fast under pressure**.

🧠 🧒 ELI5 (like 5th class)

🍳 Big idea

Dockerfile = recipe
Image = food box
Container = eating the food

🧱 Step by step

1. FROM

FROM ubuntu

👉 Choose your kitchen
(“I will cook in Ubuntu kitchen”)

---

2. RUN

RUN touch mustafa.txt

👉 Cook something and keep it inside box

RUN = do work while cooking (build time)

---

3. COPY

COPY aws.txt aws.txt

👉 Take file from your laptop → put into box

COPY = copy from your system

---

4. ADD

ADD devops.pdf devops.pdf

👉 Same as COPY but smarter

ADD = copy + download + unzip

---

5. WORKDIR

WORKDIR /mustafa

👉 Go into folder

WORKDIR = cd /mustafa

---

6. CMD

CMD yum install maven -y

👉 What happens when you open the box

CMD = run time command

---

7. ENTRYPOINT

ENTRYPOINT ["yum","install","git","-y"]

👉 Forced action

ENTRYPOINT = cannot change
CMD = can change

---

8. ARG vs ENV

ARG cloud=aws
ENV course=devops

👉 Think:

ARG = temporary (only while cooking)
ENV = permanent (stays inside box)

🧠 🧑 ELI10 (clear understanding)

🔥 Lifecycle (MOST IMPORTANT)

BUILD TIME → RUN, ARG
RUN TIME → CMD, ENTRYPOINT, ENV

🧩 Your examples explained correctly

Case 1

FROM ubuntu
RUN touch mustafa.txt

👉 When you build:

- file created inside image

👉 When you run:

- file exists

Case 2 (COPY)

COPY aws.txt aws.txt

👉 Requires:

- aws.txt must be in your local folder

👉 Result:

- file copied into image

Case 3 (ADD)

ADD devops.pdf devops.pdf

👉 Same as COPY + extra features
👉 But don’t overuse it

Case 4 (ADD URL)

ADD https://... .

👉 During build:

- file downloaded into image

Case 5 (WORKDIR mistake you had)

You wrote:

WOKDIR /mustafa

👉 ❌ Wrong spelling
👉 Correct:

WORKDIR /mustafa

👉 Then:

RUN touch harish.txt

👉 File will be created inside `/mustafa`

Case 6 (RUN vs CMD — you got confused here)

RUN

RUN yum install git -y

👉 Executes during build
👉 Git installed inside image

CMD

CMD yum install maven -y

👉 Does NOT run during build
👉 Runs only when container starts

⚠️ Important

Multiple RUN → all run
Multiple CMD → only last works

Case 7 (ENTRYPOINT vs CMD)

You tried:

ENTRYPOINT ["yum","install","git","-y"]

Then:

docker run image tree httpd

👉 Result:

ENTRYPOINT always runs
extra args get appended

👉 Final command becomes:

yum install git -y tree httpd

🔥 Key difference

CMD → default command (can override)
ENTRYPOINT → fixed command (cannot override easily)

Case 8 (ARG vs ENV — you slightly wrong)

You said:

> ENV cannot override ❌

👉 Not correct

ARG

ARG cloud=aws

👉 Only during build

Override like:

docker build --build-arg cloud=azure .

ENV

ENV course=devops

👉 Available inside container

Override like:

docker run -e course=aws image

💣 Where you went wrong (important)

❌ Typos

- `docekr` → docker
- `WOKDIR` → WORKDIR

👉 These will break builds

❌ Concept confusion

You mixed:

build time vs run time

👉 That's the root of your confusion

🧠 Final memory trick (keep this)

FROM → base OS
RUN → build time work
COPY → copy file
ADD → copy + download
WORKDIR → change folder
CMD → default run command
ENTRYPOINT → forced run command
ARG → build variable
ENV → runtime variable

🧠 ELI5 (super simple story)

🍔 Restaurant example

ENTRYPOINT = restaurant menu (fixed)
CMD = default order (can change)
docker run args = what customer says

🔴 Case 1 — CMD (your first example)

FROM centos:centos7
CMD ["yum", "install", "git", "-y"]

🧠 What happens?

Run without extra input:

docker run image9

👉 It runs:

yum install git -y

✔️ installs git

Now you did this:

docker run image9 tree httpd

👉 Docker says:

User gave new command → replace CMD

👉 Final command becomes:

tree httpd

💥 PROBLEM:

tree command may not exist → fails

🔥 Key idea

CMD = gets completely replaced

🟢 Case 2 — ENTRYPOINT (your second example)

FROM centos:centos7
ENTRYPOINT ["yum", "install", "git", "-y"]

🧠 What happens?

Run:

docker run image9 tree httpd

👉 Docker says:

ENTRYPOINT is fixed → do not replace
Extra args → add to it

👉 Final command becomes:

yum install git -y tree httpd

✔️ installs:

- git
- tree
- httpd

🔥 Key idea

ENTRYPOINT = cannot replace
arguments = added

🎯 FINAL COMPARISON (remember this)

Behavior	CMD	ENTRYPOINT
Default run	yes	yes
Override	full replace	no
Extra args	replace	append
Use case	flexible	fixed

🧠 One-line memory trick

CMD → replace
ENTRYPOINT → append

💣 Where your understanding was slightly wrong

❌ You said:

> CMD extra values not perform

👉 Not correct

✔️ It DOES perform — but:

It replaces original command

❌ You said:

> docker build image9 installs git

👉 Wrong

CMD never runs during build

👉 Only runs when container starts

⚠️ BIG mistake in your approach

You used:

ENTRYPOINT ["yum", "install", "git", "-y"]

👉 This is bad practice

Why?

Every container start → installs packages again

💥 Slow + wrong design

🧠 Real-world correct usage

✅ Good example

FROM centos:centos7
RUN yum install git -y
CMD ["bash"]

👉 Install at build time, not run time

✅ Best pattern

ENTRYPOINT ["npm"]
CMD ["start"]

👉 Runs:

npm start

👉 Override:

docker run app run dev

👉 Runs:

npm run dev
